Semester 1 – Final Report

Project Name	ShadowTrace
Faculty Supervisor Name	Dr. John Burns
Team Lead Name & ID	Said Mehraliyev 000014989
Team member #1 Name & ID • What sections did this person contribute to?	Said Mehraliyev 000014989
Team member #2 Name & ID What sections did this person contribute to?	Jamil Akbarov 000015227
Team member #3 Name & ID • What sections did this person contribute to?	Ibrahim Najafguliyev 000019888

Word count guide: ~3000

1. Executive Summary

- Overview of project goals and motivation
- Summary of Semester 1 achievements
- Preview of Semester 2 development plan

2. Team Structure and Role Allocation

- Team member profiles / specializations
- Roles and responsibilities during Semester 1
- Proposed role allocation for Semester 2
- Rationale for role assignments

3. Work Breakdown

- Major tasks identified from use cases
- Sub-task decomposition
- Task dependency discussion

4. Milestone Development and Timeline

For the purposes of milestone planning, you all work must be completed by **April 30, 2026**

- Semester 2 planned milestones – including testing (see below)
- Gantt chart or timeline projection
- Success criteria for each milestone

5. Risk Assessment and Mitigation Planning

- Risk assessment and mitigation for Semester 2
- Contingency planning

6. Final / Confirmed Selection of Tools, Technologies, and Resource Planning

- Software, hardware, and platforms needed
- Licensing, budget constraints, and procurement plan
- Infrastructure setup timeline

7. Evaluation and Validation Strategy

- Testing approaches for Semester 2, for example
    - Unit Testing
    - Integration Testing
    - System Testing
    - User Acceptance Testing (UAT)
    - Performance Testing
    - Security Testing
- Metrics for evaluating project success
- User testing plan (if applicable)

8. Ethical, Legal, and Professional Considerations

- Data protection and privacy concerns
- Ethical implications of design decisions

- Compliance and standards

9. Conclusion and Forward Strategy

- Summary of Semester 1 outcomes
- Summary Semester 2 action plan
- Long-term vision beyond the capstone

Appendices

- A. Your Research Report
- B. Your User Design
- C. Any Supplementary diagrams
- D. Preliminary code or prototypes

1. Executive Summary

Modern microservice applications rely on distributed tracing frameworks like OpenTelemetry for system observability and performance monitoring. However, default auto-instrumentation pipelines inadvertently capture and store sensitive metadata, including Personally Identifiable Information (PII), access tokens, and credentials, all in plain text. This creates a severe privacy and compliance liability under global regulations such as GDPR and CCPA. While security teams require telemetry data to detect internal threats like application-layer lateral movement, inspecting raw payloads directly violates data minimization mandates.

ShadowTrace addresses this "Privacy-Security Paradox" by decoupling threat detection from data exposure through a metadata-blind security framework. Built as a dual-stage, in-stream pipeline, ShadowTrace first sanitizes telemetry at the edge using an entropy-based Privacy Quantifier to scrub sensitive attributes. Next, it employs a Spatio-Temporal Graph Neural Network (ST-GNN) to analyze the redacted communication topologies and time-weighted call latencies over a sliding temporal window.

The project follows a two-semester research-led methodology. Semester 1 focuses on empirical privacy auditing, theoretical mathematical modeling, and offline ST-GNN algorithmic validation against established baselines (Ma, 2026). Semester 2 delivers a production-grade, in-stream OpenTelemetry Collector processor written in Golang, integrated with an Istio Service Mesh closed-loop enforcement engine for real-time automated security policy generation.

2. Team Structure Allocation

Team member profiles / specializations. The team comprises three members. The Team Lead is a software engineer with responsibility for the research and machine-learning track, working in Python with PyTorch Geometric and NetworkX. Member 2 is a software engineer focused on the telemetry pipeline, currently learning Go in preparation for the custom OpenTelemetry Collector processor. Member 3 is a cybersecurity student focused on infrastructure and threat modelling, currently learning Kubernetes.

Semester 1 responsibilities. Work follows a research-first sequence in which algorithms are validated before any production code is written. The Team Lead owns the offline graph experiments and authors Appendix A alongside the executive summary and conclusion. Member 2 owns telemetry instrumentation and trace harvesting, and documents the technology stack, evaluation strategy and project timeline. Member 3 owns the Minikube testbed, the Online Boutique deployment and the k6 attack scripts, and authors Appendix B together with the risk and ethics sections.

Semester 2 allocation. Roles shift from validation to engineering. Member 2 becomes implementation lead, translating the validated redaction and windowing logic into the Go collector processor. Member 3 becomes deployment lead, handling the Istio service-mesh integration and performance benchmarking. The Team Lead becomes integration lead, moving the detection model from notebook to inference service and consolidating both the capstone report and the research manuscript.

Rationale. Each member owns the layer matching their specialization, so learning effort compounds rather than being spent once. Documentation is assigned to whoever performs the corresponding technical work, which keeps the written record accurate. Deliberately, no member holds sole knowledge of a layer: the Team Lead reviews all three, ensuring the project survives any single member's unavailability

3. Work Breakdown

Major tasks from the use cases. Four use cases drive the work. Redaction configuration requires an entropy-based sensitivity scorer, a tiering mechanism and a versioned policy store. Alert triage requires the four detection features, a baseline comparator and a triage interface. Automated enforcement requires policy templates, a service-mesh integration and post-action verification. Compliance review requires retention controls, a coverage

calculator and an export mechanism. A fifth workstream sits outside the journeys: the offline research track that trains and benchmarks the detector.

Sub-task decomposition. The redaction workstream decomposes into entropy scoring over span attributes, computing the sensitivity score from its three signals, mapping scores to tiers, applying keyed tokenization and persisting signed policy versions. The detection workstream decomposes into span reconstruction into request graphs, windowing at sixty-second intervals, computing path entropy, causal-history distance, temporal jitter and call repetition, maintaining rolling baselines, and applying thresholds. Enforcement decomposes into class-to-template mapping, dry-run application, severity gating, expiry handling and post-enforcement re-measurement. Compliance decomposes into retention configuration, coverage aggregation and report generation.

Dependencies. The chain is strictly ordered. Redaction must precede feature extraction, because features are computed only on what survives redaction. Feature extraction must precede alerting, and alerting must precede enforcement, since enforcement acts on a classified alert. Compliance depends on all three, as its evidence is their byproduct. The critical path therefore runs from the testbed through instrumentation and harvesting to feature extraction and model validation. The offline research track depends only on the testbed, so it can proceed in parallel — which is why it is scheduled first. Enforcement has no research dependency and is deliberately last, making it the component that can be cut without invalidating the core result.

4. Milestone Development and Timeline

4.1 Gantt Chart & Timeline Projection

PROJECT MILESTONE GANTT CHART (2026 – 2027)									
Task / Milestone Phase	**Oct**	**Nov**	**Dec**	**Jan**	**Feb**	**Mar**	**Apr**	**May**	**Assigned Lead**
M1: Lab Setup & OTel Pipeline									Member #3 (K8s)
M2: k6 Generation & Privacy Audit									Member #2 (SWE)
M3: ST-GNN Model Validation (PyG)									Team Lead (ML)

M4: Semester 1 Capstone Submission									All Members
M5: In-Stream Go Collector Dev									Member #2 (SWE)
M6: Istio Mesh Closed-Loop Enforcer									Member #3 (K8s)
M7: k6 Performance Profiling (<5ms)									Member #2 (SWE)
M8: Final Defense & Code Freeze									All Members

4.2 Milestone Decomposition & Success Criteria

The development lifecycle is divided into eight discrete, measurable milestones across Semester 1 (Fall 2026) and Semester 2 (Spring 2027):

Milestone ID	Target Completion	Core Technical Deliverable	Success Criteria & Verification
M1: Lab Setup & OTel Pipeline	October 15, 2026	Deployment of Google Online Boutique on local Minikube cluster with OpenTelemetry Collector OTLP ingestion.	All microservice pods show status Running in Minikube; OTel Collector verifies OTLP ingestion on ports 4317 and 4318.
M2: Privacy Audit & Dataset Creation	November 15, 2026	Execution of k6 load scripts (normal + 5 attack scenarios) and automated raw trace harvesting (harvest_traces.py).	500+ structured raw trace JSON files collected in data/raw_traces/; Shannon Entropy audit quantifies PII leakage percentage.
M3: ST-GNN Algorithmic Validation	December 15, 2026	Offline PyTorch Geometric model training on time-weighted graph sequences (t=5s).	Model achieves multi-class classification Macro F1≥0.90 on redacted trace topology, outperforming Ma's (2026) baseline (F1=0.405).
M4: Semester 1	December 30, 2026	Compilation and submission of Semester 1 Final Report, Appendix A (Research	All required Markdown templates completed, verified

Capstone Submission		Report), and Appendix B (User Design).	by faculty supervisor, and committed to GitHub.
M5: In-Stream Go Processor Dev	February 15, 2027	Custom Golang OTel Collector processor for real-time, in-memory span attribute scrubbing and stateful windowing.	Custom Go module compiles within otelcol-contrib; unit tests confirm high-entropy attributes (H>Hthresh) redacted in memory.
M6: Istio Mesh Closed-Loop Policy	March 20, 2027	Integration with Istio Service Mesh to automatically compile and apply AuthorizationPolicy YAML manifests upon threat detection.	Automated policy generator outputs valid Istio YAML files; applied policies successfully block unauthorized lateral movement paths in Minikube.
M7: k6 Performance Profiling	April 15, 2027	End-to-end load testing under high synthetic throughput (100–1,000 req/sec) using Prometheus/Grafana monitoring.	In-stream processing overhead confirmed <5ms added latency per span; zero memory leaks in 5s sliding-window memory buffer.
M8: Final Capstone Defense	April 30, 2027	Code freeze, repository documentation, system demonstration, and final thesis defense.	Working live system demonstration, complete codebase pushed to GitHub, and successful capstone thesis defense.

4.3 Task Dependencies and Risk Buffers

The project architecture relies on clear task handoffs between team members:

- **Infrastructure to Pipeline Handoff (M1 → M2):** Member #3 (Kubernetes/Minikube) provides the running Google Online Boutique cluster, allowing Member #2 (SWE) to execute harvest_traces.py and capture the 500+ raw trace dataset.
- **Pipeline to Research Handoff (M2 → M3):** Member #2 provides the raw trace JSON files, enabling the Team Lead (Research/ML) to run the Shannon Entropy Privacy Audit and train the PyTorch Geometric ST-GNN model.
- **Research to Engineering Handoff (M3 → M5):** The mathematical algorithms validated offline in Python during Semester 1 are translated by Member #2 into production-grade Golang code inside the OpenTelemetry Collector during Semester 2.

A two-week buffer is allocated between Milestone 7 (April 15, 2027) and Milestone 8 (April 30, 2027) to handle potential performance tuning, bug fixes, and presentation rehearsal prior to the final project freeze.

5. Risk assessment and mitigation planning

Risks come from four places: the machine, the cost of watching it, the detector itself, and data protection law. Two of three features depend on the clock, so whatever disturbs timing disturbs the features. The danger: such an error looks like a result, not an error.

Local deployment. Everything runs on one Minikube node: application, collector and load generator compete for resources. Services may die mid-run, leaving a partial dataset that looks fine; such trials are discarded. Worse, a busy processor produces delays identical to an attack's, and jitter cannot tell them apart. The response: record idle timing beforehand, isolate load generation, repeat every trial, so interference shows up as inconsistency between runs. A single node hides network effects, so the deployment demonstrates feasibility and cost, not production behaviour.

Tracing overhead. Instrumentation raises the response times being measured; it can only be characterised and reported. The sharpest risk is span loss: repetition is a count, so missing spans push it silently downward, and brute-force login is exactly where a suppressed count means a missed attack. Lossy sampling is disabled, loss rates accompany every result, and lossy runs are invalid, not corrected. Path entropy fails similarly, less visibly.

Detector risks. An adversary aware of the logic can pace calls to stay beneath thresholds; results therefore describe a non-adaptive attacker. Entropy redaction misses short and legacy-format secrets, so pattern matching carries equal weight. Retraining on confirmed anomalies reinforces what the model already catches, so confirmed-benign cases are fed back with equal rigour.

Compliance. Covered in Section 8: synthetic environment, pseudonymous rather than anonymous output, dual-use scorer stated openly.

Residual position. Phases 1 and 2 establish the result on Minikube; the Phase 4 Istio integration can be cut without invalidating it. The riskiest component is deliberately the most expendable.

6. Final / Confirmed Selection of Tools, Technologies, and Resource Planning

Software, Hardware, and Platforms Needed

The execution of ShadowTrace relies on a fully open-source, cloud-native technology stack deployed locally to eliminate cloud infrastructure costs and maintain deterministic experimental conditions:

Infrastructure & Orchestration Layer: Minikube (v1.28+) running a single-node Kubernetes cluster on local x86_64 host hardware (16GB+ RAM, 8+ CPU cores, running Linux or Windows 11). Local virtualization provides complete control over network interfaces and cluster state, avoiding noisy-neighbor effects inherent in multi-tenant public cloud environments.

Target Application Benchmark: Google Online Boutique (11 polyglot microservices written in Go, C#, Node.js, Python, and Java). This benchmark simulates a production e-commerce platform with complex call paths, providing realistic inter-service dependencies, database queries, and synthetic user workflows.

Telemetry & Pipeline Infrastructure: OpenTelemetry (OTel) auto-instrumentation SDKs and the OpenTelemetry Collector (Contrib distribution v0.158.0+). Telemetry is transmitted using the vendor-neutral OpenTelemetry Protocol (OTLP) over gRPC (port 4317) and HTTP (port 4318).

Custom Pipeline Components: The in-stream Privacy Quantifier and stateful sliding-window buffer are developed in Golang (v1.22+). Go's native concurrency primitives (goroutines and channels) and zero-allocation memory patterns allow high-throughput entropy calculation without introducing significant latency to the telemetry pipeline.

Analytics & Machine Learning Engine: Python (v3.11+) utilizing PyTorch Geometric (PyG v2.5+) for Spatio-Temporal Graph Neural Network (ST-GNN) modeling, NetworkX for graph manipulation and adjacency matrix formulation, and Pandas/Scikit-learn for offline dataset handling, feature scaling, and confusion-matrix evaluation.

Visualization, Load Generation, & Policy Mesh: Jaeger for local trace topology visualization; k6 (v0.49+) for automated synthetic user and attack traffic generation; Prometheus/Grafana for scraping Envoy proxy metrics; and Istio Service Mesh (v1.20+) for executing closed-loop AuthorizationPolicy enforcement in Semester 2.

Licensing, Budget Constraints, and Procurement Plan

The project operates under a $0 procurement budget. All selected software tools and libraries are governed by permissive open-source licenses:

Apache 2.0 License: Google Online Boutique, OpenTelemetry, Golang, PyTorch, and Istio.

MIT License: NetworkX, k6, and Scikit-learn.

CNCF Governance: Kubernetes, Minikube, Jaeger, and Prometheus.

By hosting the entire deployment on local developer hardware via Minikube and avoiding proprietary cloud services (e.g., AWS, Datadog), the project incurs zero ongoing operational expenditure and remains fully reproducible across developer machines.

Infrastructure Setup Timeline

Semester 1 (Phase 1 – Completed): Local Minikube cluster spin-up, deployment of Google Online Boutique manifests, OTel Collector configuration with OTLP receivers, and verification of local trace harvesting to file storage (traces.json).

Semester 1 (Phase 2 – Current): Automated attack script development (k6), offline trace dataset generation, Python feature extraction (inter-span latency $T_{child} - T_{parent}$), and initial ST-GNN model validation against Ma (2026).

Semester 2 (Phase 3 – Planned): Compilation of the custom Golang OTel Collector processor module, deployment of in-stream Shannon Entropy redaction, and stateful windowing buffer integration.

Semester 2 (Phase 4 – Planned): Integration with Istio Service Mesh, automated YAML policy compilation, k6 end-to-end performance overhead profiling, and final paper thesis compilation.

7. Evaluation and Validation Strategy

Unit Testing: Go-level unit tests (go test) validate the internal logic of the custom OTel Collector processor. Specifically, unit tests evaluate the Shannon Entropy calculation function ($H = -\sum p(x) \log_2 p(x)$) against synthetic test strings containing known PII, JWT authorization tokens, plain-text passwords, and benign static attributes (e.g., status: 200) to confirm exact threshold boundary triggers.

Integration Testing: Verifies end-to-end trace context propagation across container boundaries. Tests confirm that microservice pods in Minikube correctly inject trace headers, that the OTel Collector receives OTLP streams on ports 4317/4318, and that sanitized span records are emitted to storage without span dropping or trace corruption.

System Testing: Tests the full pipeline under stateful operation. Evaluates the memory stability of the 5-second sliding-window buffer under continuous traffic, ensuring that late-arriving spans are correctly attached to their parent TraceID and that memory is immediately freed upon window expiration to prevent memory leaks.

User Acceptance Testing (UAT) / Operator Usability: Evaluates the output of the Telemetry Governance Engine (T_s) against security operator workflows. Tests confirm that generated Istio AuthorizationPolicy and Kubernetes NetworkPolicy YAML files are syntactically valid, human-readable, and accurately block target malicious service paths without disrupting benign user shopping workflows.

Performance Testing: Stress-tests the in-stream pipeline using k6 under synthetic load (100 to 1,000 requests per second). Measures the per-span processing latency added by the Go entropy processor (targeting <5ms added overhead) and monitors CPU/Memory utilization of the collector pod under peak load.

Security & Adversarial Testing: Evaluates model detection resilience against five simulated application-layer attack scenarios: HTTP flood, brute-force login, SQL injection probe, SSRF-like probe, and exfiltration simulation. Testing specifically evaluates evasion attempts, such as "low-and-slow" attacks where an adversary introduces artificial delays to stay beneath temporal volume thresholds.

Metrics for Evaluating Project Success

The project's success is evaluated across three quantitative axes:

Privacy Metric (Attribute Redaction Efficiency): Percentage of high-entropy credentials, tokens, and PII successfully scrubbed at the collection layer prior to storage (Target: 100% redaction of attributes exceeding the defined entropy threshold H_{thresh}).

Security Metric (Classification Fidelity): Multi-class Macro F1-score, Precision, and Recall achieved by the metadata-blind ST-GNN on redacted spatio-temporal graphs. Success is defined as achieving a Macro F1 $\geq$0.90 on redacted traces, significantly outperforming Yunjian Ma's (2026) static trace-only baseline (F1=0.405) [1].

Performance Metric (Pipeline Overhead): Latency impact introduced by in-stream processing (Target: <5ms processing overhead per span) and zero span loss under standard operational load.

User Testing Plan

Because ShadowTrace is an automated infrastructure governance framework rather than a consumer-facing application, user testing is structured around Security Operator & Developer Workflows:

Declarative Configuration Validation: Verifying that developers can declare service sensitivity tiers (D=0 to 5) via standard Kubernetes deployment labels without needing to modify application source code.

Alert Interpretability: Confirming that alerts emitted by the ST-GNN provide actionable topological context (e.g., identifying the specific source and target pods involved in an unauthorized lateral movement hop) while remaining completely free of unredacted PII.

8. Ethical, Legal and Professional Considerations

Section 1.2 explains why GDPR minimization makes this worth studying; this section asks whether it complies.

Legal framework. The GDPR governs. Article 4(1) reaches session identifiers and free-text log fields; 5(1)(c) limits collection to the necessary; 25 requires that limit built into the architecture, not retrofitted; 32 requires technical security. 5(1)(e)'s storage limitation is served by automatic expiry of observation windows, retention being a configured store property rather than housekeeping. With the researcher in Azerbaijan and venue in the EU, the stricter regime applies. The CCPA/CPRA does not bind this non-commercial project but points the same way.

Data processed. The target environment is Google Online Boutique, which generates session identifiers automatically, ships fabricated catalogue data and mocks payment. Traffic is script-generated and attacks simulated, so no natural person is identifiable and Article 4(1) does not engage. One qualification matters: the Privacy Layer replaces sensitive values with keyed HMAC tokens that stay linkable by design, which is pseudonymization under Article 4(5), and Recital 26 holds pseudonymous data to remain personal data. The position therefore rests on the traffic being synthetic, not on redaction, and would bind immediately on live deployment.

Dual use. The entropy scorer is by construction a working secret-detection engine: given telemetry, it locates credentials. It is named as such, run only against the local environment, and scoped to redaction rather than extraction. The sensitivity score's blast-radius term likewise maps which endpoints are most consequential to compromise. Inherited attack scripts run locally only; no new technique is developed.

Professional obligations. Online Boutique's Apache-2.0 terms and Ma (2026)'s release terms are verified and honoured, the inherited and original boundary kept explicit, ACM requirements met, publication committed regardless of results, and the trial-level split fixed before numbers are seen.

Ethical review. No participants, no personal data: exemption expected, confirmed before Phase 1 instrumentation.